

Homework #1

November 9, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

Read in the data and drop the `id` and `bond_id` columns which clearly have no predictive power. Those columns that have “type” in the column name and the `is_callable` column are categorical variables which should be converted into dummy variables. Any other columns are on ratio scale. There are 46 of them. After the conversion, including the dummy variables, there are 69 predictors in the data (note that the response `trade_price` is also in the data now). We then add a constant column with value 1 for intercept in the regression.

```
[1]: import pandas as pd

data = pd.read_csv('bonddata.txt', sep=',').drop(['bond_id', 'id'],
→axis='columns')

# categorical
catCols = ['is_callable'] + [col for col in data.columns if 'type' in col]

# predictors on ratio scale
rsCols = [col for col in data.columns if col not in ['is_callable',
→'trade_price'] and 'type' not in col]

data = pd.get_dummies(data=data, columns=catCols, drop_first=True)
data['constant'] = 1

print(len(rsCols))
print(data.shape)
```

46
(1620, 71)

Below we plot the histogram for the first few predictors that are on ratio scale. The skewness for some of the variables is clear

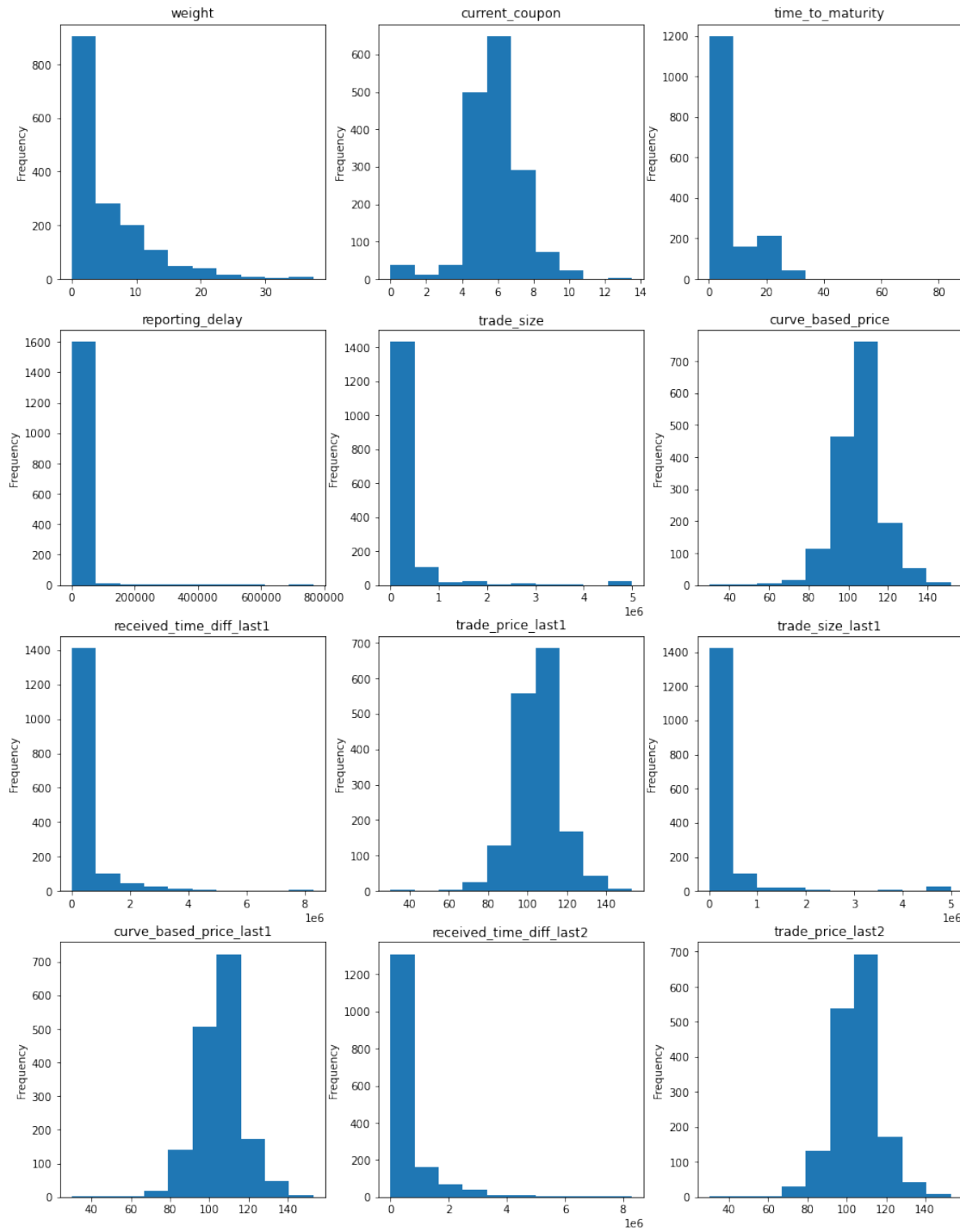
```
[2]: import matplotlib.pyplot as plt

def histplotGrid(suptitle):
    fig, axs = plt.subplots(4, 3, figsize=(15, 20))
    fig.suptitle(suptitle, size=20)

    for col, ax in zip(rsCols[:12], axs.flatten()):
        data[col].plot(kind='hist', ax=ax)
        ax.set_title(col)

histplotGrid('Histogram for Selected Predictors')
```

Histogram for Selected Predictors



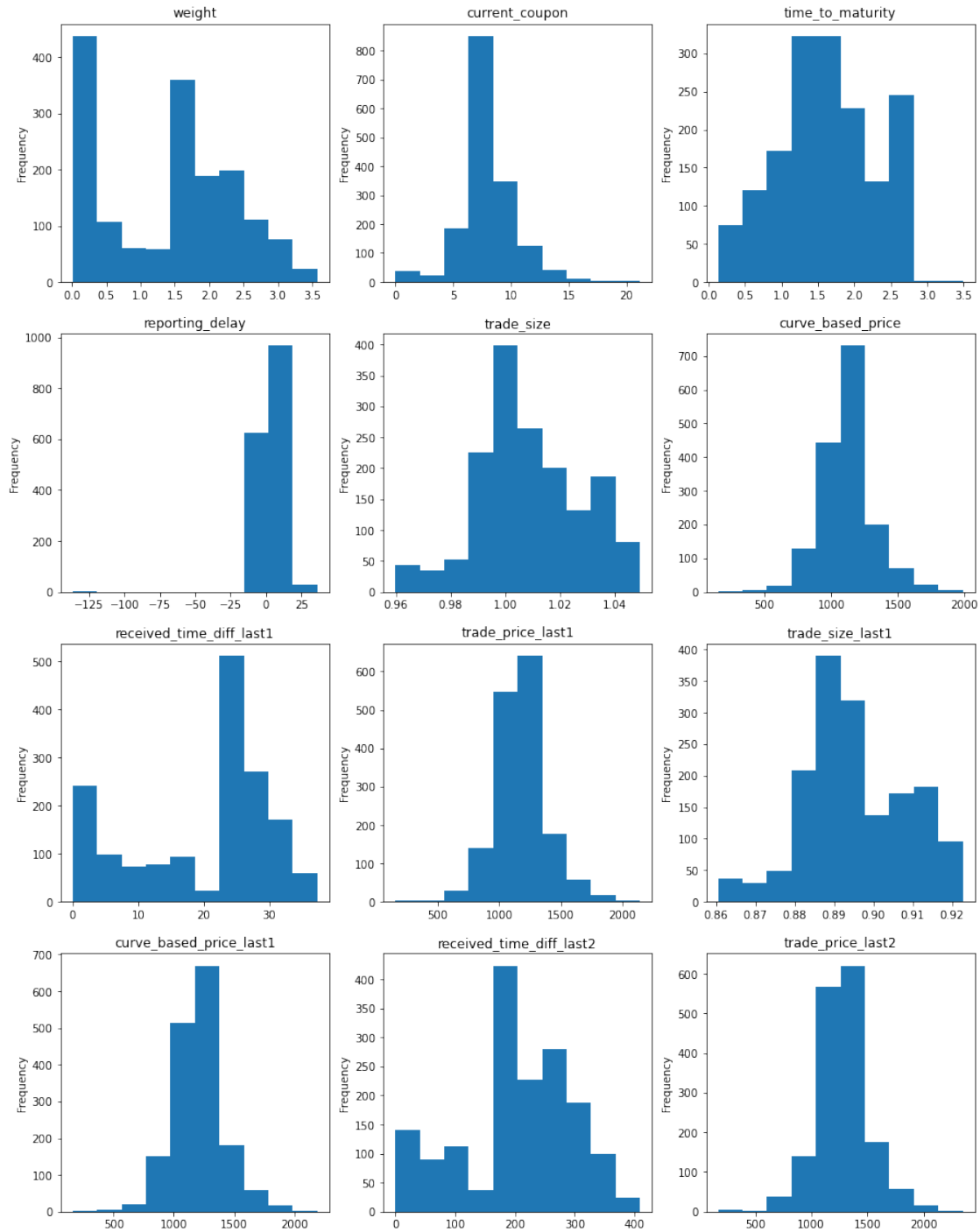
We then apply the Yeo-Johnson transformation on each of those 46 predictors. The `trade_size` columns and the `received_time_diff` columns have large values so a $\log(1 + x)$ transformation is applied beforehand. The histogram of the first few predictors after transformation are shown below.

```
[3]: from scipy import stats
import numpy as np

lambdaDict = {}
for col in rsCols:
    if 'received_time_diff' in col or 'trade_size' in col:
        data[col] = np.log(data[col] + 1)
        data[col], lambdaDict[col] = stats.yeojohnson(data[col])

histplotGrid('Histogram for Selected Predictors After the Yeo-Johnson_
↪Transformation')
```

Histogram for Selected Predictors After the Yeo-Johnson Transformation



Convert all dummy variables to float so that rpy2 can work.

```
[4]: for col in data.columns:
      if 'type' in col or 'is_callable' in col:
          data[col] = data[col].map(float)
```

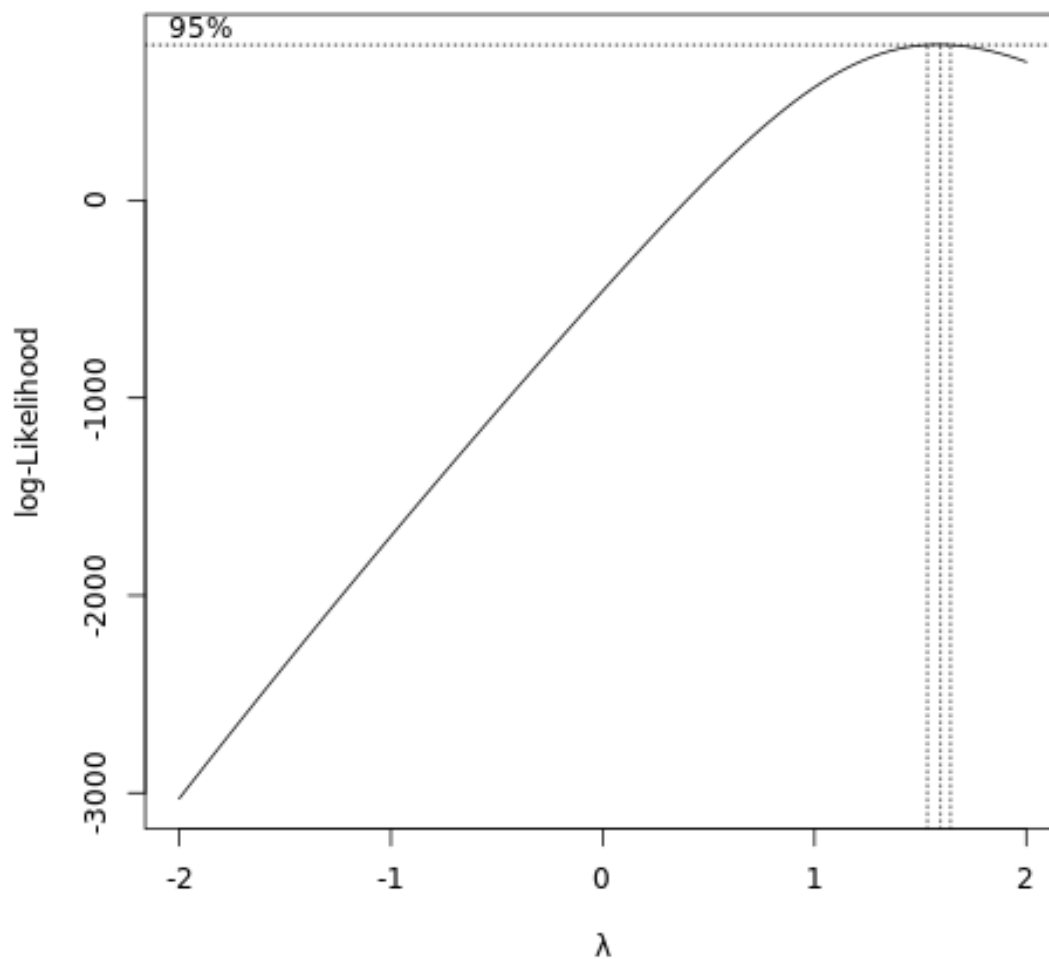
The boxcox R procedure shows that the optimal λ for the response is 1.6.

```
[5]: import rpy2
      %load_ext rpy2.ipython
```

```
[6]: %%R -i data

      library(MASS)
      res = boxcox(trade_price~., data=data)
      res$x[which(res$y==max(res$y))]
```

```
[1] 1.59596
```



Now we fit the model $y^{1.6} = \sum \beta_i x_i$. Note that, with $\lambda = 1.6$, the Box-Cox transformation $y^{(1.6)}$ is merely a linear function of $y^{1.6}$, so using either as the response is equivalent. Below are the diagnostic plots and a summary of the fitting results.

```
[7]: import statsmodels.api as sm
from scipy.stats import norm
from statsmodels.graphics.gofplots import qqplot

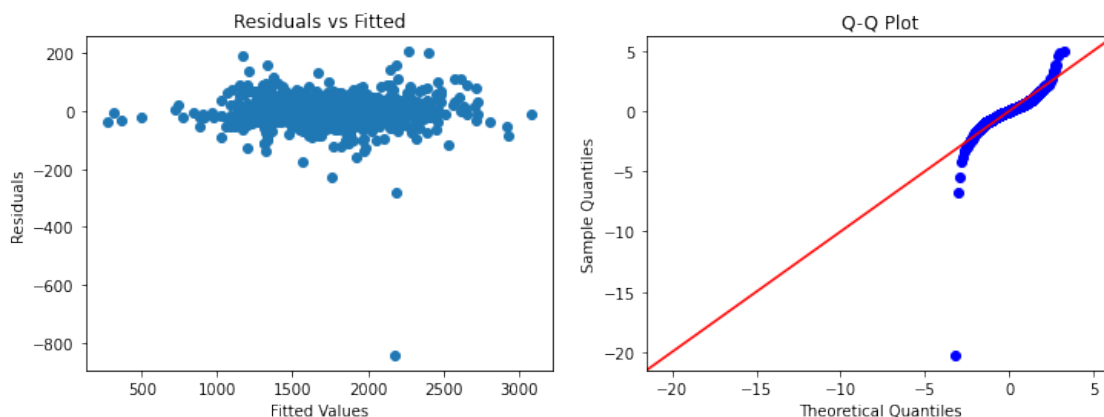
model = sm.OLS(endog=(data['trade_price']**(1.6)), exog=data.
↳drop('trade_price', axis='columns')).fit()

fig, axs = plt.subplots(1, 2, figsize=(12, 4))

axs[0].scatter(model.fittedvalues, model.resid)
axs[0].set(title='Residuals vs Fitted', xlabel='Fitted Values',
↳ylabel='Residuals')

qqplot(model.resid, dist=norm, line='45', fit=True, ax=axs[1])
axs[1].set(title='Q-Q Plot')

plt.show()
model.summary()
```



```
[7]: <class 'statsmodels.iolib.summary.Summary'>
"""
```

OLS Regression Results

```
=====
Dep. Variable:          trade_price    R-squared:                0.980
Model:                  OLS           Adj. R-squared:            0.979
```

Method:	Least Squares	F-statistic:	1121.
Date:	Sun, 08 Nov 2020	Prob (F-statistic):	0.00
Time:	23:13:22	Log-Likelihood:	-8337.0
No. Observations:	1620	AIC:	1.681e+04
Df Residuals:	1550	BIC:	1.719e+04
Df Model:	69		
Covariance Type:	nonrobust		

=====

	coef	std err	t	P> t
[0.025 0.975]				

weight	4.3275	5.236	0.827	0.409
-5.942 14.597				
current_coupon	0.4612	0.502	0.918	0.359
-0.524 1.447				
time_to_maturity	-0.3372	1.870	-0.180	0.857
-4.005 3.331				
reporting_delay	-0.6268	0.189	-3.325	0.001
-0.997 -0.257				
trade_size	413.2111	65.700	6.289	0.000
284.341 542.081				
curve_based_price	0.5760	0.063	9.135	0.000
0.452 0.700				
received_time_diff_last1	-0.2879	0.444	-0.649	0.516
-1.158 0.582				
trade_price_last1	0.6073	0.053	11.540	0.000
0.504 0.711				
trade_size_last1	-320.4019	117.097	-2.736	0.006
-550.088 -90.716				
curve_based_price_last1	0.0080	0.094	0.085	0.932
-0.177 0.193				
received_time_diff_last2	-0.0109	0.025	-0.439	0.660
-0.059 0.038				
trade_price_last2	0.0474	0.052	0.911	0.363
-0.055 0.150				
trade_size_last2	72.2001	134.662	0.536	0.592
-191.938 336.338				
curve_based_price_last2	-0.0873	0.086	-1.015	0.310
-0.256 0.081				
received_time_diff_last3	0.0067	0.010	0.709	0.478
-0.012 0.025				
trade_price_last3	0.0711	0.056	1.277	0.202
-0.038 0.180				
trade_size_last3	-67.0124	112.123	-0.598	0.550
-286.941 152.917				

curve_based_price_last3	-0.1626	0.092	-1.766	0.078
-0.343 0.018				
received_time_diff_last4	-0.0032	0.006	-0.505	0.613
-0.016 0.009				
trade_price_last4	0.0984	0.042	2.332	0.020
0.016 0.181				
trade_size_last4	-110.5800	120.450	-0.918	0.359
-346.843 125.683				
curve_based_price_last4	0.2949	0.104	2.829	0.005
0.090 0.499				
received_time_diff_last5	-0.0023	0.006	-0.398	0.691
-0.014 0.009				
trade_price_last5	0.0872	0.063	1.393	0.164
-0.036 0.210				
trade_size_last5	-33.9659	90.184	-0.377	0.707
-210.861 142.930				
curve_based_price_last5	-0.4627	0.123	-3.759	0.000
-0.704 -0.221				
received_time_diff_last6	-0.0016	0.006	-0.277	0.782
-0.013 0.010				
trade_price_last6	-0.0471	0.041	-1.137	0.256
-0.128 0.034				
trade_size_last6	59.3817	76.222	0.779	0.436
-90.127 208.891				
curve_based_price_last6	0.2194	0.112	1.962	0.050
8.15e-05 0.439				
received_time_diff_last7	0.0038	0.005	0.836	0.404
-0.005 0.013				
trade_price_last7	0.1195	0.040	3.015	0.003
0.042 0.197				
trade_size_last7	-76.7364	115.842	-0.662	0.508
-303.959 150.486				
curve_based_price_last7	-0.0823	0.128	-0.642	0.521
-0.334 0.169				
received_time_diff_last8	0.0056	0.005	1.160	0.246
-0.004 0.015				
trade_price_last8	0.0237	0.046	0.520	0.603
-0.066 0.113				
trade_size_last8	344.6340	245.867	1.402	0.161
-137.633 826.901				
curve_based_price_last8	-0.1841	0.138	-1.339	0.181
-0.454 0.086				
received_time_diff_last9	-0.0018	0.003	-0.564	0.573
-0.008 0.005				
trade_price_last9	0.0932	0.052	1.780	0.075
-0.010 0.196				
trade_size_last9	-282.6446	200.502	-1.410	0.159

-675.929	110.640				
curve_based_price_last9		0.0488	0.133	0.366	0.714
-0.212	0.310				
received_time_diff_last10		-0.0015	0.002	-0.874	0.382
-0.005	0.002				
trade_price_last10		0.0467	0.050	0.930	0.353
-0.052	0.145				
trade_size_last10		98.5492	135.225	0.729	0.466
-166.694	363.792				
curve_based_price_last10		0.0822	0.108	0.764	0.445
-0.129	0.293				
is_callable_1		3.3541	3.597	0.932	0.351
-3.701	10.409				
trade_type_3		42.5159	2.959	14.366	0.000
36.711	48.321				
trade_type_4		23.7003	2.717	8.722	0.000
18.370	29.030				
trade_type_last1_3		-12.3164	3.261	-3.777	0.000
-18.712	-5.921				
trade_type_last1_4		-7.1272	2.953	-2.414	0.016
-12.919	-1.336				
trade_type_last2_3		-2.3736	3.239	-0.733	0.464
-8.726	3.979				
trade_type_last2_4		-0.1689	2.852	-0.059	0.953
-5.764	5.426				
trade_type_last3_3		-3.5840	3.201	-1.120	0.263
-9.863	2.695				
trade_type_last3_4		-1.2002	2.898	-0.414	0.679
-6.884	4.484				
trade_type_last4_3		-10.4868	3.284	-3.194	0.001
-16.927	-4.046				
trade_type_last4_4		-5.5536	2.839	-1.956	0.051
-11.122	0.014				
trade_type_last5_3		-5.8175	3.234	-1.799	0.072
-12.161	0.526				
trade_type_last5_4		-1.5781	2.914	-0.541	0.588
-7.294	4.138				
trade_type_last6_3		0.8869	3.373	0.263	0.793
-5.730	7.504				
trade_type_last6_4		1.7649	2.909	0.607	0.544
-3.941	7.471				
trade_type_last7_3		-2.1018	3.244	-0.648	0.517
-8.464	4.261				
trade_type_last7_4		1.3685	2.895	0.473	0.637
-4.311	7.048				
trade_type_last8_3		0.8777	3.175	0.276	0.782
-5.349	7.105				

trade_type_last8_4	2.4514	2.833	0.865	0.387
-3.106	8.009			
trade_type_last9_3	-3.9902	3.148	-1.267	0.205
-10.166	2.185			
trade_type_last9_4	-5.1828	2.846	-1.821	0.069
-10.765	0.400			
trade_type_last10_3	0.6712	3.076	0.218	0.827
-5.363	6.705			
trade_type_last10_4	1.7351	2.747	0.632	0.528
-3.653	7.123			
constant	-75.6304	150.458	-0.503	0.615
-370.754	219.493			

Omnibus:	1868.799	Durbin-Watson:	1.981
Prob(Omnibus):	0.000	Jarque-Bera (JB):	769785.296
Skew:	-5.247	Prob(JB):	0.00
Kurtosis:	109.274	Cond. No.	4.33e+06

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.33e+06. This might indicate that there are strong multicollinearity or other numerical problems.

"""

2.

Below is the new `treg()` function. The differences from the one in lecture are 1. In `neglogliketreg()`, ν is included in the parameter list in the first argument 1. The optimization procedure now tries to optimize ν too, for which an initial guess $\nu = 15$ is used 1. The optimization procedure uses the L-BFGS-B method so that we have the flexibility to specify $\sigma > 0$, $\nu > 0$ 1. `numdifftools.Hessian` is used to estimate the variance of the MLE

```
[1]: from scipy import stats
from scipy.optimize import minimize
import numpy as np
import numdifftools as nd

def treg(x, y):
    def neglogliketreg(params, x, y):
        beta0, beta1, sigma, nu = params
        resids = y - (x*beta1 + beta0)
        return (-1)*stats.t.logpdf(resids/sigma, nu).sum() + len(x)*np.
        ↪ log(sigma)
```

```

holdcov = np.cov(x, y)
beta1init = holdcov[0, 1]/holdcov[0, 0]
beta0init = np.mean(y) - np.mean(x)*beta1init
resids = y - (beta0init + beta1init*x)
sigmainit = np.sqrt((resids**2).sum()/len(x))
nuinit = 15

mleout = minimize(neglogliketreg, [beta0init, beta1init, sigmainit,
↪nuinit], args=(x, y), method='L-BFGS-B', bounds=[(None, None), (None, None),
↪(0.001, None), (0.001, None)])
hessfunc = nd.Hessian(neglogliketreg)

return mleout.x, np.linalg.inv(hessfunc(mleout.x, x, y))

```

An experiment with a sample of size 10,000 confirms that the implementation is correct.

```

[14]: from pandas import DataFrame

beta0 = 4
beta1 = 5
sigma = 10
nu = 7

n = 10000
x = np.random.rand(n)
y = beta0 + beta1*x + sigma*stats.t.rvs(nu, size=n)
mle, var = treg(x, y)
z = stats.norm.ppf(1-0.025)

print('MLE for beta0, beta1, sigma and nu: ', mle)
print()
print('Covariance matrix of the MLE: \n', DataFrame(var))
print()
print('Confidence Intervals for the 4 parameters: ')
[(mle[i] - z*np.sqrt(var[i][i]), mle[i] + z*np.sqrt(var[i][i])) for i in
↪range(4)]

```

```

MLE for beta0, beta1, sigma and nu: [ 4.18376003  4.75342245 10.06324371
7.07164504]

```

Covariance matrix of the MLE:

	0	1	2	3
0	0.051483	-0.076462	0.000183	0.000856
1	-0.076462	0.150574	0.000541	0.002535
2	0.000183	0.000541	0.014199	0.039475
3	0.000856	0.002535	0.039475	0.223160

Confidence Intervals for the 4 parameters:

```
[14]: [(3.7390453810256306, 4.628474672287717),
      (3.9928811570170923, 5.51396374734906),
      (9.829695539886353, 10.29679188761744),
      (6.145760793144649, 7.997529284715806)]
```

Finally we generate the confidence intervals multiple times to confirm that 95% of the time the true value of the parameters are covered.

```
[17]: nSim = 1000

beta0 = 4
beta1 = 5
sigma = 10
nu = 7

beta0CIs = []
beta1CIs = []
sigmaCIs = []
nuCIs = []

for i in range(nSim):
    n = 10000
    x = np.random.rand(n)
    y = beta0 + beta1*x + sigma*stats.t.rvs(nu, size=n)
    mle, var = treg(x, y)

    beta0CIs.append((mle[0] - z*np.sqrt(var[0][0]), mle[0] + z*np.
→sqrt(var[0][0])))
    beta1CIs.append((mle[1] - z*np.sqrt(var[1][1]), mle[1] + z*np.
→sqrt(var[1][1])))
    sigmaCIs.append((mle[2] - z*np.sqrt(var[2][2]), mle[2] + z*np.
→sqrt(var[2][2])))
    nuCIs.append((mle[3] - z*np.sqrt(var[3][3]), mle[3] + z*np.sqrt(var[3][3])))

beta0InCI = sum([1 for l, r in beta0CIs if l < beta0 < r])
beta1InCI = sum([1 for l, r in beta1CIs if l < beta1 < r])
sigmaInCI = sum([1 for l, r in sigmaCIs if l < sigma < r])
nuInCI = sum([1 for l, r in nuCIs if l < nu < r])

print(f'{beta0InCI} out of {nSim} experiments generated a confidence interval_
→that covers {beta0}')
print(f'{beta1InCI} out of {nSim} experiments generated a confidence interval_
→that covers {beta1}')
print(f'{sigmaInCI} out of {nSim} experiments generated a confidence interval_
→that covers {sigma}')
print(f'{nuInCI} out of {nSim} experiments generated a confidence interval that_
→covers {nu}')
```

960 out of 1000 experiments generated a confidence interval that covers 4
966 out of 1000 experiments generated a confidence interval that covers 5
958 out of 1000 experiments generated a confidence interval that covers 10
956 out of 1000 experiments generated a confidence interval that covers 7

3.

We will use `scipy.optimize.minimize` to find the MLE for $\text{Beta}(\alpha, \beta)$ numerically, with MOM estimators as the initial guess. Below we find the MOM estimators for $\text{Beta}(\alpha, \beta)$ first.

Let $\hat{m}_1$ and $\hat{m}_2$ be the sample first and second moments, that is $\hat{m}_1 = \sum_{i=1}^n X_i/n$, $\hat{m}_2 = \sum_{i=1}^n X_i^2/n$. The MOM estimators $\hat{\alpha}_{\text{MOM}}$, $\hat{\beta}_{\text{MOM}}$ are the solution to the system of equations

$$\begin{cases} \hat{m}_1 &= \frac{\alpha}{\alpha+\beta} \\ \hat{m}_2 &= \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)} + \left(\frac{\alpha}{\alpha+\beta}\right)^2, \end{cases}$$

where the right hand side are the first two moments of $\text{Beta}(\alpha, \beta)$. To solve the system, first note that $1 - \hat{m}_1 = \beta/(\alpha + \beta)$ and hence

$$(\hat{m}_2 - \hat{m}_1^2)(\alpha + \beta + 1) = \frac{\alpha\beta}{(\alpha + \beta)^2} = \hat{m}_1(1 - \hat{m}_1).$$

Also from the first equation we have $\hat{m}_1(\alpha + \beta) = \alpha$. Thus we obtain a linear system of α and β :

$$\begin{cases} (\hat{m}_2 - \hat{m}_1^2)(\alpha + \beta + 1) &= \hat{m}_1(1 - \hat{m}_1) \\ \hat{m}_1(\alpha + \beta) &= \alpha \end{cases}.$$

A little algebra shows that the MOM estimators are

$$\begin{aligned} \hat{\alpha}_{\text{MOM}} &= \frac{\hat{m}_1^2(1 - \hat{m}_1)}{\hat{m}_2 - \hat{m}_1^2} - \hat{m}_1, \\ \hat{\beta}_{\text{MOM}} &= \frac{1 - \hat{m}_1}{\hat{m}_1} \hat{\alpha}_{\text{MOM}}. \end{aligned}$$

These are the initial guess for the optimization procedure we will be using below.

Below we define a function to compute the MLE and its variance numerically.

```
[6]: from scipy.optimize import minimize
from scipy import stats
import numdifftools as nd
import numpy as np

def betamle(x):
    def negloglikelihood(params, x):
        a, b = params
        return (-1)*stats.beta.logpdf(x, a, b).sum()
```

```

m1 = x.mean()
m2 = (x**2).mean()
ahatmom = (m1**2)*(1-m1)/(m2-m1**2) - m1
bhatmom = ahatmom*(1-m1)/m1

mleout = minimize(negloglikelihood, [ahatmom, bhatmom], args=(x),
↳method='L-BFGS-B', bounds=[(0.001, None), (0.001, None)])
hessfunc = nd.Hessian(negloglikelihood)

return mleout.x, np.linalg.inv(hessfunc(mleout.x, x))

```

Now we run simulation to confirm the above implementation is correct. First generate a sample of size 10,000 from the Beta(5,3) distribution. We see that the estimates by the above numerical procedure are quite close to the true values $\alpha = 5, \beta = 3$.

```

[11]: n = 10000
x = stats.beta(a=5, b=3).rvs(n)

mle, var = betamle(x)

print(mle)
print(var)

```

```

[5.00418234 3.01174226]
[[0.00489035 0.00249623]
 [0.00249623 0.00165838]]

```

Given the estimated covariance matrix of the MLE, the 95% confidence interval of both parameters can be found as below.

```

[12]: z = stats.norm.ppf(1 - 0.025)

print('95% Confidence Interval for alpha: ', (mle[0] - z*np.sqrt(var[0][0]),
↳mle[0] + z*np.sqrt(var[0][0])))
print('95% Confidence Interval for beta: ', (mle[1] - z*np.sqrt(var[1][1]),
↳mle[1] + z*np.sqrt(var[1][1])))

```

```

95% Confidence Interval for alpha: (4.867119992230483, 5.141244679178911)
95% Confidence Interval for beta: (2.9319261522355946, 3.0915583756979492)

```

Finally we generate the confidence intervals multiple times to confirm that 95% of the time the true value of the parameters are covered.

```

[29]: nSim = 1000

a = 5
b = 3
alphaCIs = []
betaCIs = []

```

```

for i in range(nSim):
    n = 10000
    x = stats.beta(a=a, b=b).rvs(n)
    mle, var = betamle(x)

    alphaCIs.append((mle[0] - z*np.sqrt(var[0][0]), mle[0] + z*np.
→sqrt(var[0][0])))
    betaCIs.append((mle[1] - z*np.sqrt(var[1][1]), mle[1] + z*np.
→sqrt(var[1][1])))

aInCI = sum([1 for l, r in alphaCIs if l<a<r])
bInCI = sum([1 for l, r in betaCIs if l<b<r])

print(f'{aInCI} out of {nSim} experiments generated a confidence interval that_
→covers {a}')
print(f'{bInCI} out of {nSim} experiments generated a confidence interval that_
→covers {b}')

```

947 out of 1000 experiments generated a confidence interval that covers 5
949 out of 1000 experiments generated a confidence interval that covers 3